

Por que usar um framework?

Do JavaScript puro ao React

Se eu já sei JavaScript, por que preciso de mais alguma coisa?

O que você vai conseguir fazer

1

Ler e escrever

componentes simples em React

2

Explicar estado

e como ele controla a interface

3

Comparar abordagens

JS puro e framework, sem dogma

4

Trabalhar em equipe

com entregas pequenas e incrementais

Agenda · três blocos

Bloco 1

Conceitos Diagnóstico e vocabulário: estado, evento, componente, reatividade.

Bloco 2

Demonstrações A mesma tela em JS puro e em React, ao vivo, com previsão de bugs.

Bloco 3

Prática Atividade em trios (30 min), fechamento e política de IA.

Sondagem individual – sem nota

- Responda sozinho, em 6 minutos.
- Não vale nota e não é corrigido publicamente.
- Serve para ajustar o ritmo das próximas aulas.
- "Não sei ainda" é uma resposta legítima.

Objetivo: mapear o ponto de partida da turma, não classificar pessoas.

Quatro perguntas abertas

1. O que acontece na tela quando você clica em um botão?

2. Você já alterou o conteúdo de uma página com JavaScript? Como?

3. O que você entende por "componente"?

4. Já ouviu falar em React, Vite ou Node? O que faz cada um?

Escala de autopercepção (1 a 5)

Lógica em JavaScript (variáveis, funções, arrays)

1 2 3 4 5

Manipular a página com o DOM

1 2 3 4 5

Escrever HTML e CSS

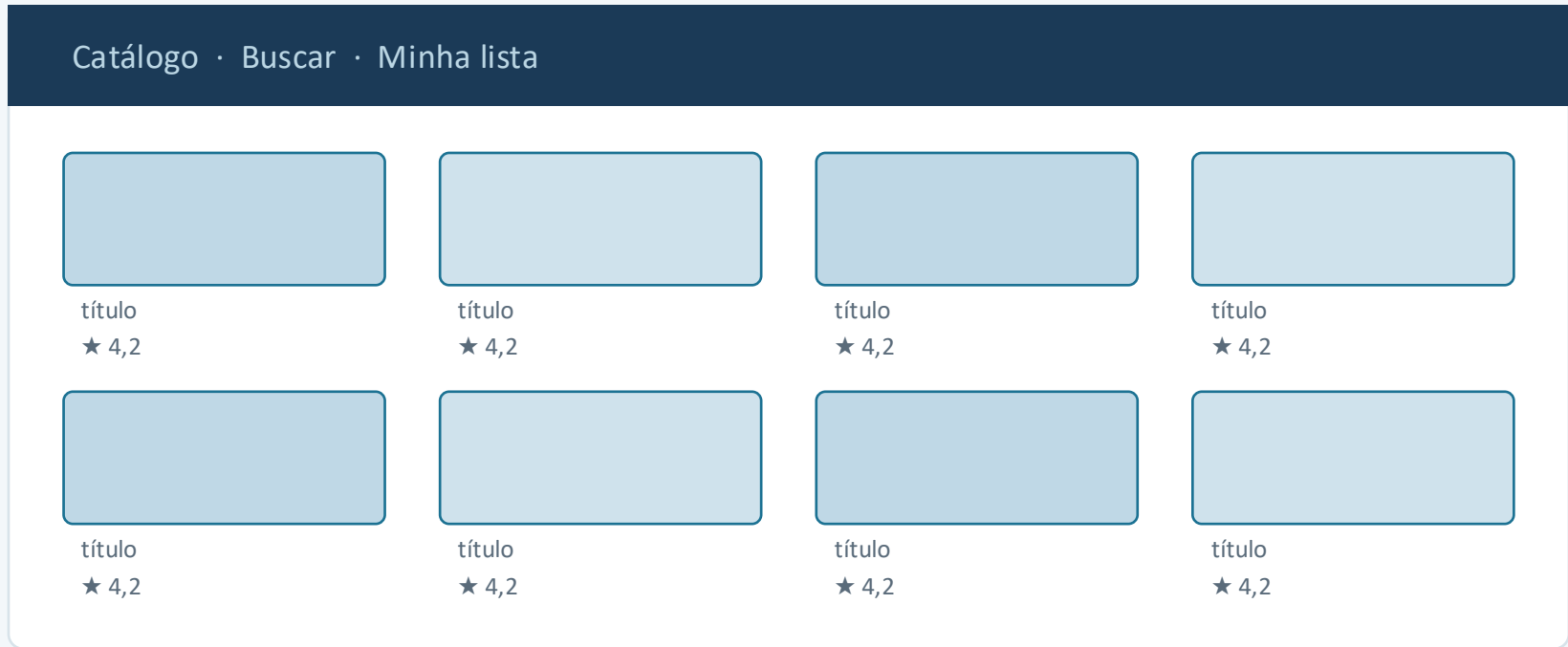
1 2 3 4 5

Usar terminal e instalar pacotes

1 2 3 4 5

1 = nunca fiz · 5 = faço com autonomia

Uma tela é feita de partes repetidas



O mesmo cartão, 8 vezes — muda só o conteúdo.

O que muda nesta tela?

E o que permanece igual?

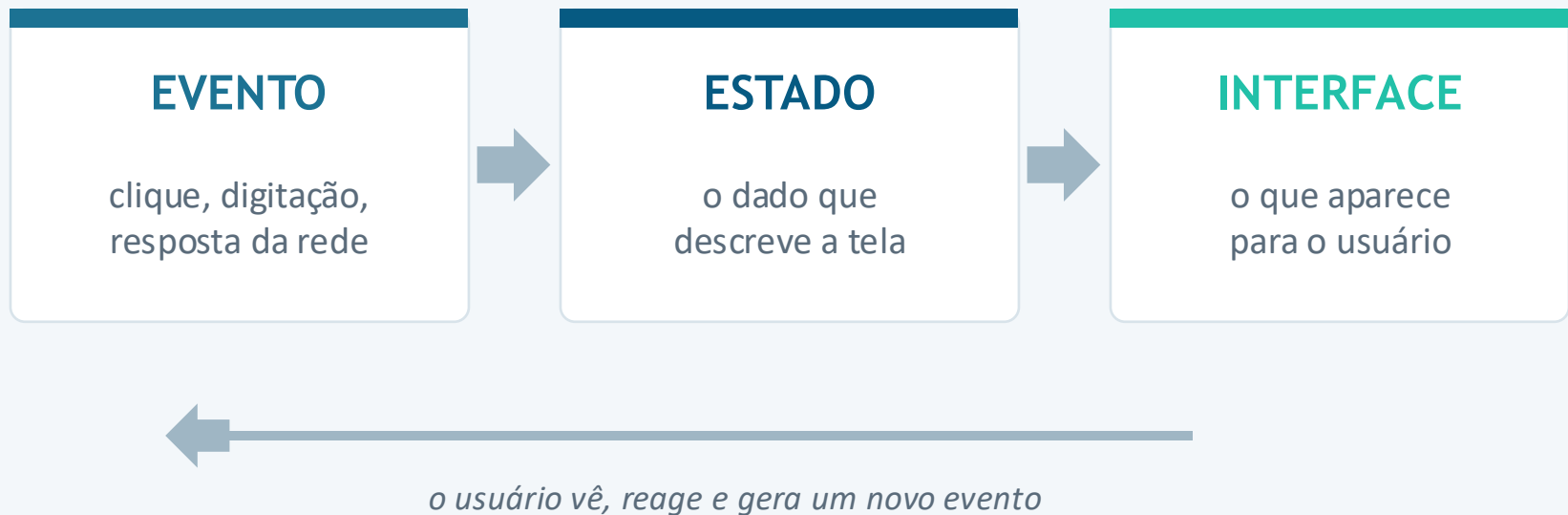
coração vazio → cheio

lista filtrada pela busca

contador de itens

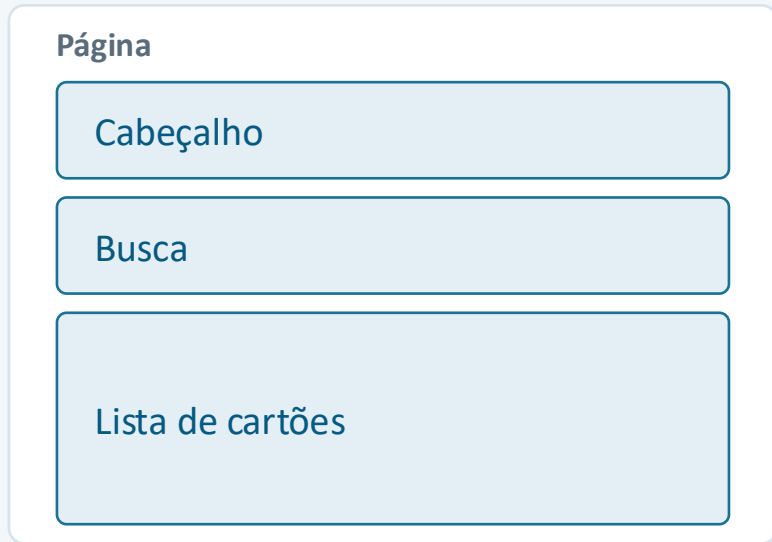
mensagem "nada encontrado"

Evento → Estado → Interface



Componentização

Quebrar a tela em pedaços nomeados e reutilizáveis.



- Cada pedaço tem um nome.
- Cada pedaço recebe dados de fora.
- O mesmo pedaço é reutilizado várias vezes.
- Erros ficam localizados em um lugar só.

Estado

O dado que descreve a tela neste instante.

Contador

`contagem = 0`



`contagem = 8`

Busca

`texto = ""`



`texto = "jazz"`

Curtida

`curtido = falso`



`curtido = verdadeiro`

Mudou o estado → a tela precisa mudar junto.

Reatividade

interface = $f(\text{estado})$

Sem reatividade

- muda o dado
- você atualiza a tela
- você lembra de tudo

Com reatividade

- muda o dado
- a tela acompanha
- você descreve o resultado

Planilha: mude a célula e a fórmula recalcula sozinha.

Biblioteca · Framework · Ferramenta

Biblioteca

Você chama o código dela quando quer.

Você mantém o controle do fluxo.

Framework

Ele chama o seu código nos pontos previstos.

Ele define a estrutura do projeto.

Ferramenta de build

Prepara o código para rodar no navegador.

Não aparece na tela do usuário.

Nenhuma das três é "melhor": elas resolvem problemas diferentes.

React, Vite e Next.js – quem faz o quê

React

Biblioteca de interface. Você descreve componentes e estado.

Usaremos em todo o curso.

Vite

Ferramenta de build e servidor de desenvolvimento. Roda o projeto rápido.

Usaremos como base.

Next.js

Framework construído sobre React, com rotas e renderização no servidor.

Só citado neste curso.

DEMONSTRAÇÃO AO VIVO

Contador em JavaScript puro

Sem biblioteca. Só HTML, um botão e o DOM. · ~10 minutos

Acompanhe: onde está o estado? quem atualiza a tela?

Três responsabilidades no código

```
let contagem = 0;
const alvo =
document.getElementById("valor");
const botao =
document.getElementById("mais");

botao.addEventListener("click", () => {
  contagem = contagem + 1;
  alvo.textContent = contagem;
});
```



Estado

a variável guarda o dado



Evento

o clique dispara a ação



Atualização do DOM

você escreve na tela

PREVEJA O BUG · 1 minuto em duplas

O estado muda, mas a tela não muda.

Código

```
contagem = contagem + 1;  
// alvo.textContent = contagem;
```

O que a turma prevê?

A memória avança: 1, 2, 3...

A tela continua em 0.

Quem falhou? A ponte entre dado e tela — feita à mão.

DEMONSTRAÇÃO AO VIVO

O mesmo contador em React

Um componente, um estado, nenhuma escrita manual no DOM. · ~10 minutos

Acompanhe: o que desapareceu em relação à versão anterior?

Componente funcional com useState

```
function Contador() {  
  const [contagem, setContagem] = useState(0);  
  
  return (  
    <button  
      onClick={() => setContagem(contagem +  
1)}>  
      Cliques: {contagem}  
    </button>  
  );  
}
```



Componente

uma função que descreve um pedaço da tela



Estado

valor atual + função que o troca



Descrição

a tela é escrita em função do estado

Mesma aplicação, duas abordagens

Aspecto	JavaScript puro	React
Estado	variável que você mantém	useState no componente
Atualizar a tela	you escreve no DOM	descrita em função do estado
Repetir blocos	copiar ou gerar por laço	reutilizar o componente
Custo inicial	nenhum: abre o arquivo	instalar e configurar o projeto
Brilha quando	a página é pequena	a interface cresce e se repete

Dois ferramentas legítimas. A escolha depende do problema.

USO DE IA E INTEGRIDADE ACADÊMICA

Pode

explicar erros, sugerir caminhos, revisar seu raciocínio

Não pode

entregar código que você não sabe explicar

Sempre

declare no README quando e como usou IA

Se a interface é uma função do estado, o que precisamos dominar em JavaScript para descrevê-la?